



Finance Manager

FRONT END

BACK END

DESCRIPTION

Finance Manager is a simple, secure app that helps individuals and small teams plan, track, and understand their money in one place. It brings together budgeting, income and expense tracking, cash-flow visibility, and financial reporting so you can make informed decisions faster.

Key features typically include:

- Budget planning with categories and spending limits
- Transaction tracking (manual entry or bank import, depending on setup)
- Bill reminders and recurring payments
- Goals and savings tracking
- Cash-flow and balance insights with dashboards and charts
- Reports for monthly performance, categories, and trends
- Export options (CSV/PDF) for accounting, tax, or sharing

Designed for day-to-day use, the app highlights where your money is going, flags overspending early, and helps you stay on top of upcoming obligations—so you can manage finances with clarity and confidence.

ARCHITECTURE (MUST READ)

FINANCE-MANAGER/

— app/	# ● PRESENTATION & DELIVERY LAYER (Next.js)
— core/	# ● DOMAIN LAYER (Shared App-Wide)

|— features/ # ● APPLICATION & INFRASTRUCTURE (Vertical Slices)
|— utils/ # ● INFRASTRUCTURE (Global configurations)

app/ (Presentation & Delivery)

Should be treated **strictly as the Presentation and Delivery Layer**. It handles routing, rendering UI, and receiving HTTP requests.

- **(dashboard)/** : Next.js pages and layouts (e.g., `transactions/page.tsx`). These components call Use Cases, never Supabase directly.
- **api/** : Delivery layer for external API routes (e.g., `stock/history/route.ts`).

core/ (Shared Domain)

Contains globally shared business rules, types, and utilities that do not depend on any specific framework or feature.

- **entities/** : App-wide types (e.g., Base Error types, Result wrappers).
- **formatters/** : Pure functions for data transformation (e.g., Currency formatting).

features/ (Application & Infrastructure)

This is where the actual business logic lives, isolated into vertical slices by feature (e.g., `transactions/` , `investments/`). Inside each feature:

- **domain/** : Feature-specific types and interfaces (e.g., `types.ts`).
- **use-cases/** : The core business logic and workflows (e.g., `GetUserTransactions.ts`)
- **infrastructure/** : The specific implementation of data fetching for this feature (e.g., `SupabaseTransactionRepository.ts`).

utils/ (Global Infrastructure)

Houses global, third-party integrations and configurations.

- **supabase/** : The raw Supabase setup (`client.ts` , `server.ts` , `middleware.ts`).